

React Machine Test: Product Dashboard + Cart System

Objective

Build a responsive React application with product listing, search, filter, cart management, order summary, and basic user dashboard screens.

The candidate should create clean UI, reusable components, proper state management, API/data handling, and responsive layout.

Screen 1: Login Screen

Purpose

User should be able to log in before accessing the dashboard.

UI Sections

Left Side

- App name/logo
- Short welcome text
Example: "Welcome back to ShopAdmin"

Right Side

Login form with:

- Email input
- Password input
- Login button
- Error message area

Functionality

- Validate email and password
- Email should be required and valid
- Password should be minimum 6 characters
- On successful login, redirect to Dashboard
- Store login status in localStorage

- Show error if credentials are empty or invalid

Expected Concepts

- Form handling
 - Validation
 - Controlled components
 - LocalStorage
 - Conditional routing
-

Screen 2: Dashboard Screen

Purpose

Show quick overview after login.

UI Sections

Header

- App name
- Search bar
- User name/profile icon
- Logout button

Sidebar

Menu items:

- Dashboard
- Products
- Cart
- Orders
- Profile

Dashboard Cards

Show 4 summary cards:

- Total Products
- Cart Items
- Total Cart Value
- Total Orders

Recent Activity Section

Show simple list:

- Product added to cart
- Product removed
- Order placed

Functionality

- Show dynamic numbers based on data
- Logout should clear localStorage and redirect to Login
- Sidebar navigation should work

Expected Concepts

- Layout design
 - React Router
 - State sharing
 - Reusable components
 - Protected route
-

Screen 3: Product Listing Screen

Purpose

Display product list with search, filter, and sorting.

UI Sections

Top Area

- Page title: Products
- Search input
- Category filter dropdown
- Sort dropdown

Product Grid

Each product card should show:

- Product image
- Product name
- Category

- Price
- Rating
- Stock status
- Add to Cart button

Empty State

If no product is found, show: "No products found."

Functionality

- Fetch products from dummy API or local JSON
- Search by product name
- Filter by category
- Sort by price: Low to High / High to Low
- Add product to cart
- Disable Add to Cart button if already added
- Show loading and error state

Expected Concepts

- API integration
 - useEffect
 - Search/filter/sort logic
 - Conditional rendering
 - Component reuse
-

Screen 4: Product Details Screen

Purpose

Show detailed information for a selected product.

UI Sections

Product Image Section

- Large product image
- Optional thumbnail images

Product Info Section

- Product name
- Category
- Price

- Rating
- Availability
- Description
- Add to Cart button
- Back to Products button

Functionality

- Open details page using product ID
- Fetch or find product details based on route parameter
- Add product to cart from details page
- If product already exists in cart, show “Already in Cart”
- Back button should return to product listing

Expected Concepts

- Dynamic routing
 - Route params
 - Data lookup
 - Conditional button state
 - Component communication
-

Screen 5: Cart Screen

Purpose

Manage selected products before checkout.

UI Sections

Cart Item List

Each cart row/card should show:

- Product image
- Product name
- Price
- Quantity controls: + / -
- Item subtotal
- Remove button

Cart Summary

Show:

- Subtotal
- Discount
- Tax
- Final Total
- Checkout button

Empty Cart State

If cart is empty, show: "Your cart is empty." Button: "Continue Shopping"

Functionality

- Increase/decrease quantity
- Quantity should not go below 1
- Remove item from cart
- Calculate subtotal dynamically
- Apply fixed discount or percentage discount
- Calculate tax
- Store cart data in localStorage
- Continue Shopping button should go to Products page

Expected Concepts

- State management
- Array update logic
- LocalStorage persistence
- Derived calculations
- Reusable cart components

Screen 6: Order / Checkout Screen

Purpose

Place a simple order using cart data.

UI Sections

Customer Information Form

Fields:

- Full Name
- Email
- Phone
- Address
- City
- Pincode

Order Summary

Show:

- Cart items
- Quantity
- Total price
- Tax
- Final amount

Action Buttons

- Place Order
- Back to Cart

Functionality

- Validate all required fields
- Email should be valid
- Phone should be numeric
- Pincode should be numeric
- On order placement:
- Show success message
- Clear cart
- Save order in localStorage
- Redirect to Dashboard or Orders page

Expected Concepts

- Form validation
- Cart data usage
- LocalStorage
- Success state handling
- Navigation after submit

Bonus Screen: Orders Screen

Purpose

Show previous placed orders.

UI Sections

- Order ID
- Customer name
- Order date
- Total amount
- Order status
- View Details button

Functionality

- Read orders from localStorage
- Show empty state if no orders exist
- Optional: order details modal

Suggested Data Structure

```
const products = [  
  {  
    id: 1,  
    name: "Wireless Headphone",  
    category: "Electronics",  
    price: 2500,  
    rating: 4.5,  
    stock: 12,  
    image: "image-url",  
    description: "High quality wireless headphone with noise cancellation."  
  },  
  {  
    id: 2,  
    name: "Running Shoes",  
    category: "Fashion",  
    price: 3200,  
    rating: 4.2,  
    stock: 8,  
    image: "image-url",  
  }  
]
```

```
    description: "Comfortable running shoes for daily workout."
  }
];
```

Candidate Submission Requirements

The candidate should submit:

- GitHub repository link
- Live deployed link, if possible
- README file with setup instructions
- Clean folder structure
- Responsive UI
- Proper component naming
- No console errors

Evaluation Criteria

Area	Marks
UI design and responsiveness	15
Component structure	15
React Router implementation	10
State management	15
Search, filter, sort logic	15
Cart calculation logic	15
Form validation	10
Code readability	5

Total: 100 marks